TECHNICAL SPECIFICATION & PROJECT SYNOPSIS

AUTOSAR HLD Document Analysis Assistant

AI-Powered Retrieval-Augmented Generation (RAG) System with Two-Stage Cross-Encoder Re-Ranking for Automotive High-Level Design Verification

Target Domain: Automotive Embedded Software Systems	**Architecture Standard:** AUTOSAR Classic / Adaptive Platform
Core Frameworks: Streamlit, FAISS CPU, Sentence Transformers	**Inference Modes:** Extractive Dossier & Multi-LLM (Local / Cloud)
Supported LLMs: Ollama (DeepSeek/LLaMA), OpenRouter, OpenAI, Claude	**Target Input:** Text-based Automotive HLD / Architecture PDFs

Executive Summary: The AUTOSAR HLD Document Analysis Assistant is an advanced engineering decision-support system built to streamline the review, analysis, and verification of complex automotive High-Level Design (HLD) specifications. Utilizing PyMuPDF for page-level extraction, sliding-window chunking, FAISS vector similarity retrieval, and a deep Cross-Encoder re-ranking model, the system delivers verifiable, evidence-backed answers with strict source-page citations. Operating in both zero-API Extractive Evidence mode and Augmented LLM mode (supporting local Ollama models like DeepSeek-R1 and cloud providers like OpenRouter, OpenAI, and Claude), the tool automates component inventory extraction, interface port mapping, signal tracing, inconsistency identification, and multi-version document comparison—transforming hours of manual specification auditing into near-instantaneous synthesis.

1. Problem Statement & Automotive Engineering Motivation

Modern vehicle electronic control units (ECUs)—ranging from Body Control Modules (BCM) and Gateway Controllers to Powertrain and ADAS processors—rely on intricate AUTOSAR software architectures. High-Level Design (HLD) documents formalize Software Components (SWCs), Runtime Environment (RTE) ports, CAN/LIN/Ethernet signal mappings, Basic Software (BSW) stacks, and safety parameters. In practice, automotive software engineers face critical challenges:

- **Manual Review Overhead:** HLDs regularly exceed hundreds of dense pages. Cross-referencing interface dependencies, signal routes, and diagnostic event mappings across documents is labor-intensive and error-prone.
- **AI Hallucination Hazards:** Generic commercial LLMs frequently hallucinate automotive identifiers (e.g., fabricating non-existent AUTOSAR ports or inventing CAN IDs), creating unacceptable technical risk.
- **Data Privacy & IP Protection:** Strict OEM/Tier-1 confidentiality mandates prohibit uploading proprietary automotive ECU specifications to external third-party public AI endpoints.
- **Specification Evolution Drift:** Tracking architectural additions, deprecations, and parameter modifications between iterative document versions (e.g., v2.1 vs. v2.2) is historically manual and fragmented.

2. Project Objectives & Engineering Scope

Project Objective	Target Metric / Quality Gate	Technical Implementation
Evidence-Grounded Q&A;	100% cited answers with doc & page #	Two-stage RAG (FAISS + Cross-Encoder) with strict schema verification.
Air-Gapped & Offline Ready	100% functional without internet or API keys	Local extractive dossier synthesis + private Ollama backend execution.
Automated Entity Extraction	Tabular inventory of SWCs, Ports, Signals & BSW	Deterministic AUTOSAR-specific regex heuristics and frequency counters.
Document Version Diffing	Term addition/removal change tracking	Set-theoretic term delta extraction across multiple indexed document versions.
Inconsistency Detection	Undefined acronyms & duplicated sections	Heuristic abbreviation audits across page collections with evidence logging.

3. System Architecture & Two-Stage Retrieval Workflow

The system incorporates an end-to-end modular pipeline engineered to guarantee high recall, precision re-ranking, and deterministic response structure:

Workflow Stage	Primary Module	Technical Mechanism & Processing Logic
1. Ingestion & Validation	pdf_processor.py (PyMuPDF / fitz)	Extracts text page-by-page, verifies %PDF- headers, filters low-character non-extractable pages (<10 chars), flags scanned PDFs, preserves exact 1-based page numbers.
2. Sliding-Window Chunking	chunker.py (TextChunker)	Segments pages into 800-character segments with 150-character overlap. Assigns deterministic chunk provenance IDs (<doc>_p<page>_c<idx>) and character bounds.
3. Vector Embedding & Index	embeddings.py vector_store.py	Generates 384-dimensional normalized vectors using all-MiniLM-L6-v2 (or SHA-256 MockEmbedder). Builds FAISS IndexFlatIP for cosine similarity searching.
4. Candidate Recall (Stage 1)	rag_pipeline.py (Vector Recall)	Retrieves top-12+ candidate chunks based on inner-product vector score, ensuring broad thematic candidate recall before applying compute-intensive cross-attention.
5. Re-Ranking (Stage 2)	reranker.py (Cross-Encoder)	Scores query-passage pairs via ms-marco-MiniLM-L-6-v2 (or fallback lexical scorer with AUTOSAR term boosts). Re-orders candidates by semantic relevance to top-5.
6. Answer Synthesis	llm_provider.py (Schema Formatter)	Enforces a strict 3-section output (Executive Summary, Architectural Breakdown, Evidence Table). Strips reasoning traces (<think>) and verifies citations.

Two-Stage Re-Ranking Advantage: Standard bi-encoder vector embeddings compress entire passages into a single vector, often losing critical architectural distinctions between similarly named elements (e.g., DoorLockPort vs. WindowControlPort). The Cross-Encoder evaluates full cross-attention over all token pairs simultaneously, computing deep semantic relevance. The UI tracks rank migrations (e.g., initial rank #4 promoted to re-ranked rank #1), displaying exact score metrics to the engineer.

4. Core Software Modules & Functional Responsibilities

Module File	Class / Functions	Key Architectural Role & Responsibilities
pdf_processor.py	PDFProcessor PageRecord	Validates PDF headers, extracts page texts, detects non-extractable/image pages, preserves 1-based page metadata.
chunker.py	TextChunker TextChunk	Sliding-window text segmenter; tracks start/end character offsets and provenance IDs.
vector_store.py	VectorStore	FAISS IndexFlatIP management; handles vector serialization and metadata pickle persistence.
reranker.py	ReRanker FallbackLexicalReranker	Cross-encoder re-ranking; features fallback scoring with AUTOSAR CamelCase keyword boosts.
llm_provider.py	LLMProvider format_and_verify_answer	Multi-LLM client (Ollama/OpenRouter/OpenAI/Claude/Extractive); strips reasoning tags and verifies citations.
architecture_analyzer.py	ArchitectureAnalyzer	Deterministic regex extraction of SWCs, interfaces, signals, flows; detects undefined abbreviations.
document_comparator.py	DocumentComparator	Set-theoretic multi-document comparison; categorizes common, added, and removed terms into DataFrames.
app.py	Streamlit Dashboard	Interactive 4-tab web application (Upload & Process, Ask Questions, Architecture Analysis, Document Compare).

5. Verification, Benchmark Evaluation & Quality Assurance

To rigorously validate retrieval accuracy, citation integrity, and safety guards, the repository includes a standardized evaluation harness (evaluate.py) running 10 test categories against synthetic AUTOSAR Body Control Module (BCM) HLD specifications, alongside a complete 77-test pytest suite:

ID	Evaluation Query Category	Expected Architectural Targets	Metric Assessed	Outcome
Q01	Component Identification	LightingManager, DoorLockController, WindowController	Recall of core SWC terms in retrieved chunks	**PASS (100%)**
Q02	Interface Relationships	ExteriorLightPort, InteriorLightPort	Port-to-component linkage verification	**PASS (100%)**
Q03	Signal Definitions	LightStatusSignal, LockStatusSignal, WindowStatusSignal	Communication matrix signal recall	**PASS (100%)**
Q04	Module Dependencies	KeyFobSignalReceiver, AuthenticationService	Dependency statement extraction	**PASS (100%)**
Q05	Functional Flows	KeyFob RF → DoorLockController → CAN Status	Multi-step operational sequence tracing	**PASS (100%)**
Q06	BSW Stack Configuration	MCAL, PDUR, DCM, DEM, NVM drivers	Basic Software infrastructure mapping	**PASS (100%)**
Q07	CAN Network Signals	CAN IDs 0x301, 0x302, periodic transmission	Identifier & baud timing extraction	**PASS (100%)**
Q08	Document Architecture Summary	BCM, Body Control Module, Apex-ECU-300	High-level system synthesis quality	**PASS (100%)**
Q09	Missing Specifications / Gaps	ClimateControl fault handling, NVM roll-back	Known limitation & TBD detection	**PASS (100%)**
Q10	Unsupported Question Guard	Irrelevant query ('Stock price of AUTOSAR GmbH')	Zero false citations; rejection safety	**PASS (100%)**

6. Technology Stack & Architectural Rationale

Stack Component	Library / Package	Version / Specification	Engineering Selection Rationale
Web UI Platform	Streamlit	≥ 1.28.0	Rapid Python reactive UI; handles multi-file upload, session caching, and popover evidence views.
PDF Document Engine	PyMuPDF (fitz)	≥ 1.23.0	Ultra-fast C++ MuPDF backend; extracts native digital text and font metadata with zero OCR delay.
Dense Vector Index	FAISS (faiss-cpu)	≥ 1.7.4	Sub-millisecond cosine similarity search on CPU; scales easily to hundreds of indexed pages.
Bi-Encoder Model	sentence-transformers	all-MiniLM-L6-v2 (384-d)	Compact (~90 MB) embedding model balancing fast CPU inference with solid technical semantics.
Cross-Encoder Re-Ranker	sentence-transformers	ms-marco-MiniLM-L-6-v2	Deep query-passage cross-attention; boosts top-k precision for technical acronyms.
Local LLM Server	Ollama	HTTP API (v1)	Enables 100% offline private execution of DeepSeek-R1 (1.5B/8B) and LLaMA 3.2 models.
Cloud LLM Provider	OpenRouter / OpenAI	openai SDK ≥ 1.3.0	Single API key accessing Claude 3.5 Sonnet, GPT-4o, and Gemini 2.0 Flash models.
Automated Testing	pytest / ReportLab	≥ 7.4.0 / ≥ 4.0.0	Offline synthetic PDF creation and regression testing without requiring internet access.

7. User Interface Workflow & Windows 1-Click Operations

The application is structured around a streamlined four-tab workflow accessible via modern web browsers:

- **Tab 1: Upload & Process:** Multi-file drag-and-drop ingestion with validation checks for text extractability, page counts, and scanned image alerts.
- **Tab 2: Ask Questions:** Interactive Q&A; chat with pre-filled AUTOSAR architectural prompt buttons, popover inspection of retrieved chunks showing initial vs. re-ranked rank shifts, and ground-truth citations.
- **Tab 3: Architecture Analysis:** Automated heuristic inventory displaying candidate SWCs, interfaces, dependencies, and inconsistency alerts, complete with one-click CSV export.
- **Tab 4: Document Compare:** Side-by-side technical terminology diffing across multiple uploaded document releases.
- **1-Click Windows Batch Scripts:** start_app.bat automatically checks and starts the local Ollama LLM service, clears port conflicts, and launches Streamlit at http://localhost:8501; stop_app.bat gracefully terminates all processes.

8. Operational Boundaries & Future Development Roadmap

Current Operational Boundaries	Targeted Future Enhancements (Roadmap)
• **Scanned Document Constraint:** Only digital text PDFs are supported; image-only/scanned documents require OCR preprocessing. • **Single-Session FAISS Store:** Optimized for desktop engineering review; multi-tenant isolation requires vector database migration. • **Engineering Review Advisory:** System outputs are heuristic candidates and decision aids, not certified compliance sign-offs.	1. **ARXML System Description Integration:** Direct parsing and cross-referencing of AUTOSAR XML models with PDF specifications. 2. **Vision-Language Diagram Extraction:** Multi-modal extraction of architectural sequence and component block diagrams. 3. **Streaming Token UI:** Token-by-token streaming for reduced perceived response latency on large local models. 4. **Compliance Matrix Export:** Automated generation of ISO 26262 and ASPICE traceability matrices in ReqIF/Excel.

9. Conclusion

The AUTOSAR HLD Document Analysis Assistant bridges the gap between state-of-the-art Artificial Intelligence and mission-critical automotive software engineering. By uniting two-stage dense vector retrieval, cross-attention re-ranking, and strict ground-truth citation verification, the platform resolves AI hallucination risks. Offering dual-mode operation—from zero-cost, air-gapped local execution via Extractive Mode and Ollama to advanced cloud LLM synthesis—the system establishes an agile, verifiable, and privacy-preserving tool for automotive systems architects, safety managers, and engineering audit teams.

Project Title: AUTOSAR HLD Analysis Assistant	**Release Version:** v0.1.0-Engineering Prototype
Evaluation Pass Rate: 100% (10/10 Benchmarks | 77/77 Unit Tests)	**Compliance Status:** Engineering Research / Prototype Review
Source Repository: d:/autosar-hld-assistant	**Publication Date:** October 2026